EN3160 - Assignment 2

230266B - Jathunwaththa J.C.R.N.
GitHub Repository

[ ]:
```
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import maximum_filter
```

Question 01

[2]:
```
def scaled_log_kernel(sigma):
    radius = int(np.ceil(3 * sigma))
    coords = np.arange(-radius, radius + 1)

    x, y = np.meshgrid(coords, coords)
    r2 = x**2 + y**2

    kernel = (r2 - 2 * sigma**2) / (2 * np.pi * sigma**4) * np.exp(-r2 / (2 *␣
 ↪sigma**2))
    kernel -= kernel.mean()

    return kernel.astype(np.float32)


def find_scale_space_maxima(scale_space, sigmas, threshold):
    neighborhood = np.ones((3, 3, 3), dtype=np.uint8)
    local_max = maximum_filter(scale_space, size=(3, 3, 3), mode="nearest")
    is_max = ((scale_space == local_max) & (scale_space > threshold))

    is_max[0, :, :] = False
    is_max[-1, :, :] = False

    ks, ys, xs = np.where(is_max)
    candidates = [(x, y, sigmas[k], scale_space[k, y, x]) for k, y, x in␣
 ↪zip(ks, ys, xs)]

    return candidates
```

[ ]:
```
# Detect blob-like structures using the Laplacian of Gaussian across scales.
im = cv.imread(
    './a2images/the_berry_farms_sunflower_field.jpeg',
    cv.IMREAD_REDUCED_COLOR_4
)

gray = cv.cvtColor(im, cv.COLOR_BGR2GRAY)
```

```
sigmas = np.arange(0.5, 10, 0.5)
gray_float = gray.astype(np.float32) / 255.0
responses = []

for sigma in sigmas:
    kernel = scaled_log_kernel(sigma)
    response = cv.filter2D(
        gray_float,
        cv.CV_32F,
        kernel,
        borderType=cv.BORDER_REFLECT
    )
    responses.append(np.abs(response))

scale_space = np.stack(responses, axis=0)
candidates = find_scale_space_maxima(scale_space, sigmas, threshold=0.29)
print("Number of candidate blobs:", len(candidates))
```

```
Number of candidate blobs: 66
```

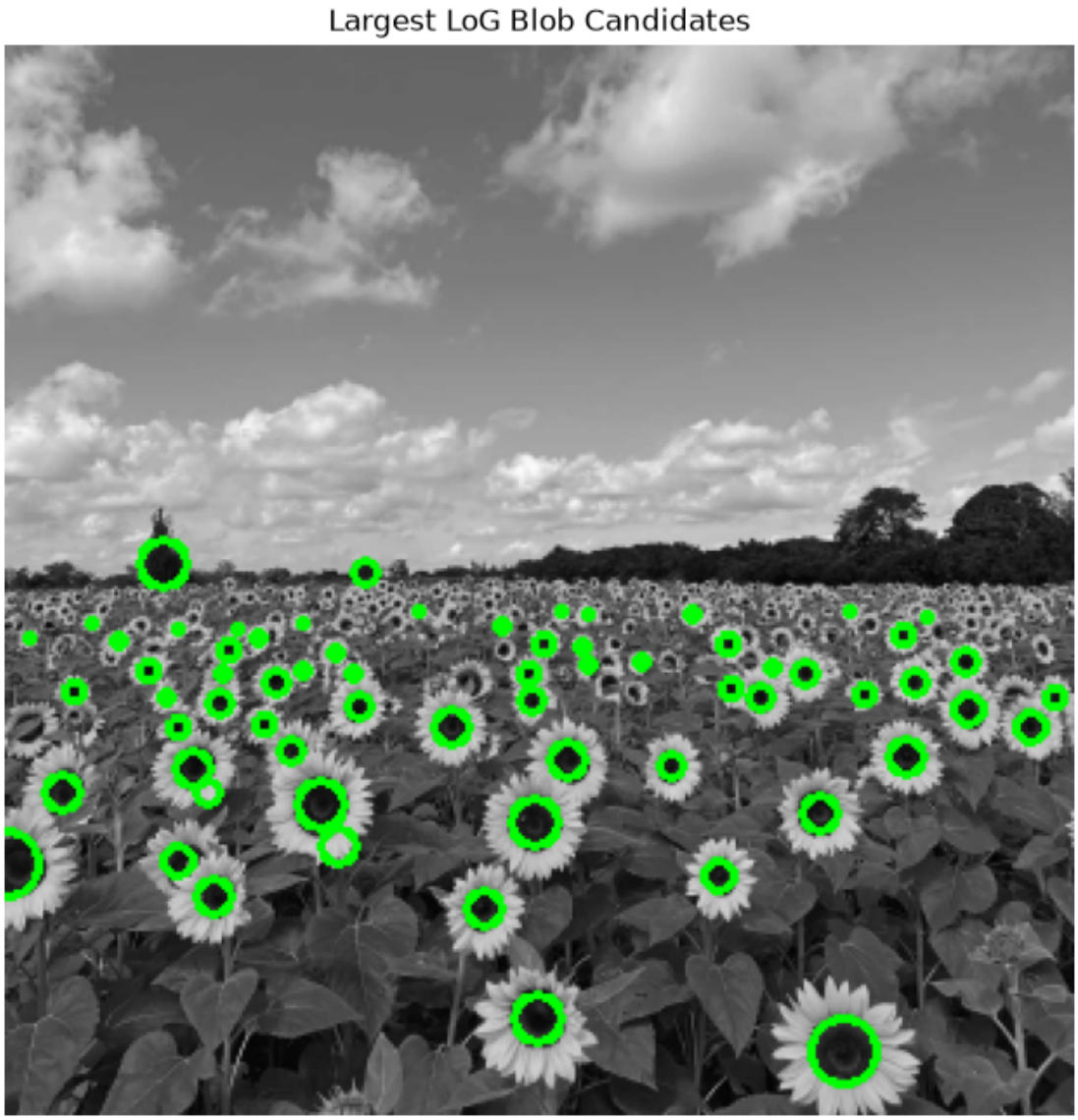
[4]:
```
result = cv.cvtColor(gray, cv.COLOR_GRAY2BGR)

# Sort by estimated radius, largest first
candidates.sort(key=lambda c: c[2], reverse=True)

for x, y, sigma, response in candidates:
    radius = int(round(np.sqrt(2) * sigma))
    cv.circle(result, (x, y), radius, (0, 255, 0), 2)

plt.figure(figsize=(12, 8))
plt.imshow(cv.cvtColor(result, cv.COLOR_BGR2RGB))
plt.title("Largest LoG Blob Candidates")
plt.axis("off")
plt.show()

# Parameters of the largest circle
if candidates:
    x, y, sigma, response = candidates[1]
    radius = int(round(np.sqrt(2) * sigma))
    print(f"Largest circle: center=({x}, {y}), radius={radius},␣
 ↪response={response}")
```




```
Largest circle: center=(282, 338), radius=11, response=0.35212352871894836
```

Parameters of the largest circle: - Center: (282, 338) - Radius: 11 - Response: 0.3521

σ values used: 0.5, 1.0, 1.5, 2.0, … ,10.0

Question 02

Data generation

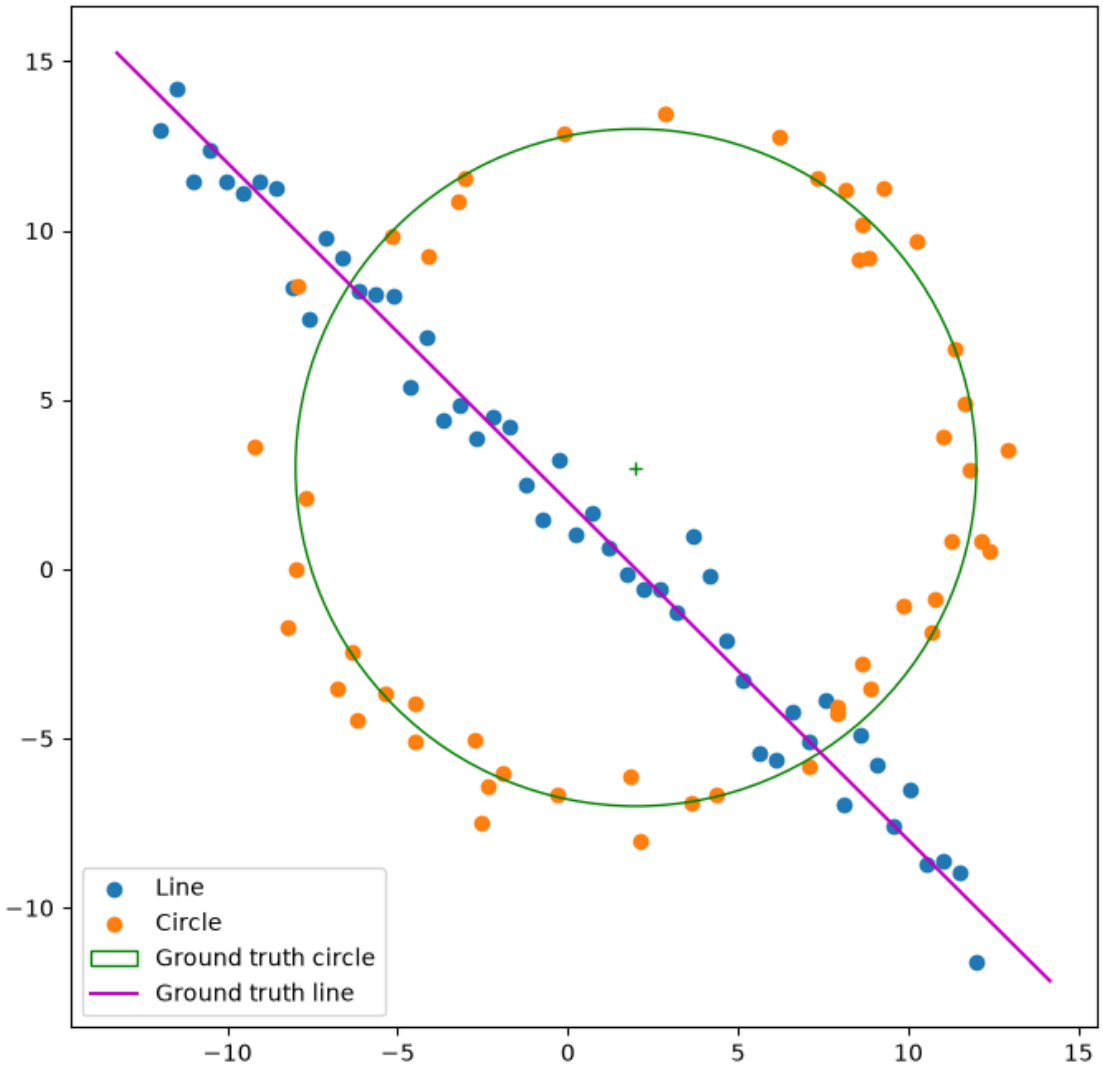
[ ]:
```
# Generate a synthetic dataset with one line and one noisy circle.
import numpy as np
from scipy.optimize import minimize
from scipy import linalg
```

```
import matplotlib.pyplot as plt

# np.random.seed(0)
N = 100
half_n = N//2
r = 10
x0_gt, y0_gt = 2, 3  # Center
s = r/16
t = np.random.uniform(0, 2*np.pi, half_n)
n = s*np.random.randn(half_n)
x, y = x0_gt + (r + n)*np.cos(t), y0_gt + (r + n)*np.sin(t)
X_circ = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))
s = 1.
m, b = -1, 2
x = np.linspace(-12, 12, half_n)
y = m*x + b + s*np.random.randn(half_n)
X_line = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))
X = np.vstack((X_circ, X_line))  # All points

fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(X_line[:, 0], X_line[:, 1], label='Line')
ax.scatter(X_circ[:, 0], X_circ[:, 1], label='Circle')
circle_gt = plt.Circle((x0_gt, y0_gt), r, color='g', fill=False, label='Ground␣
 ↪truth circle')
ax.add_patch(circle_gt)
ax.plot((x0_gt), (y0_gt), '+', color='g')
x_min, x_max = ax.get_xlim()
x_ = np.array([x_min, x_max])
y_ = m*x_ + b
plt.plot(x_, y_, color='m', label='Ground truth line')
plt.legend()
```

[ ]: <matplotlib.legend.Legend at 0x2c6f027bcd0>



Fitting the line

[ ]:
```
# RANSAC line fitting with a least-squares refit on the inlier set.
num_iterations = 1000
threshold = 1.5
min_inliers = 30
line_inlier_mask = None

best = {'count': 0, 'err': np.inf}

for _ in range(num_iterations):
    i, j = np.random.choice(len(X), size=2, replace=False)
    p1, p2 = X[i], X[j]
    dx, dy = p2 - p1
```

```
    norm = np.hypot(dx, dy)
    if norm < 1e-9:
        continue

    nx, ny = -dy / norm, dx / norm
    d = nx * p1[0] + ny * p1[1]

    dist = np.abs(X @ np.array([nx, ny]) - d)
    mask = dist < threshold
    if mask.sum() < min_inliers:
        continue

    # Total least squares refit on the consensus set
    pts = X[mask]
    centroid = pts.mean(axis=0)
    _, _, Vt = np.linalg.svd(pts - centroid)
    nx, ny = Vt[1]
    d = nx * centroid[0] + ny * centroid[1]

    # Re-evaluate the refitted line on all points
    dist = np.abs(X @ np.array([nx, ny]) - d)
    mask = dist < threshold
    count = mask.sum()
    err = np.mean(dist[mask] ** 2)

    if count > best['count'] or (count == best['count'] and err < best['err']):
        best = dict(count=count, err=err, a=nx, b=ny, d=d, mask=mask)
        line_inlier_mask = mask

a_best, b_best, d_best = best['a'], best['b'], best['d']
```

Fitting the circle

[ ]:
```
# Remove the line inliers and fit a circle to the remaining points.
X_remaining = X[~line_inlier_mask]

num_iterations = 1000
threshold = 1.5
min_inliers = 30

best = {'count': 0, 'err': np.inf}

for _ in range(num_iterations):
    i, j, k = np.random.choice(len(X_remaining), size=3, replace=False)
    p1, p2, p3 = X_remaining[i], X_remaining[j], X_remaining[k]
    # Fit a circle to the three points
    A = np.array([
```

```
        [2*(p2[0]-p1[0]), 2*(p2[1]-p1[1])],
        [2*(p3[0]-p1[0]), 2*(p3[1]-p1[1])]
    ])
    b_vec = np.array([
        p2[0]**2 + p2[1]**2 - p1[0]**2 - p1[1]**2,
        p3[0]**2 + p3[1]**2 - p1[0]**2 - p1[1]**2
    ])
    try:
        x0, y0 = np.linalg.solve(A, b_vec)
    except np.linalg.LinAlgError:
        continue
    r = np.hypot(p1[0]-x0, p1[1]-y0)

    dist = np.abs(np.hypot(X_remaining[:,0]-x0, X_remaining[:,1]-y0) - r)
    mask = dist < threshold
    if mask.sum() < min_inliers:
        continue

    count = mask.sum()
    err = np.mean(dist[mask] ** 2)

    if count > best['count'] or (count == best['count'] and err < best['err']):
        best = dict(count=count, err=err, x0=x0, y0=y0, r=r, mask=mask)

x0_best, y0_best, r_best = best['x0'], best['y0'], best['r']


# Plot ground truth and best fitting shapes
fig, ax = plt.subplots(1, 1, figsize=(8, 8))

ax.scatter(X_line[:, 0], X_line[:, 1], label='Line')
ax.scatter(X_circ[:, 0], X_circ[:, 1], label='Circle')

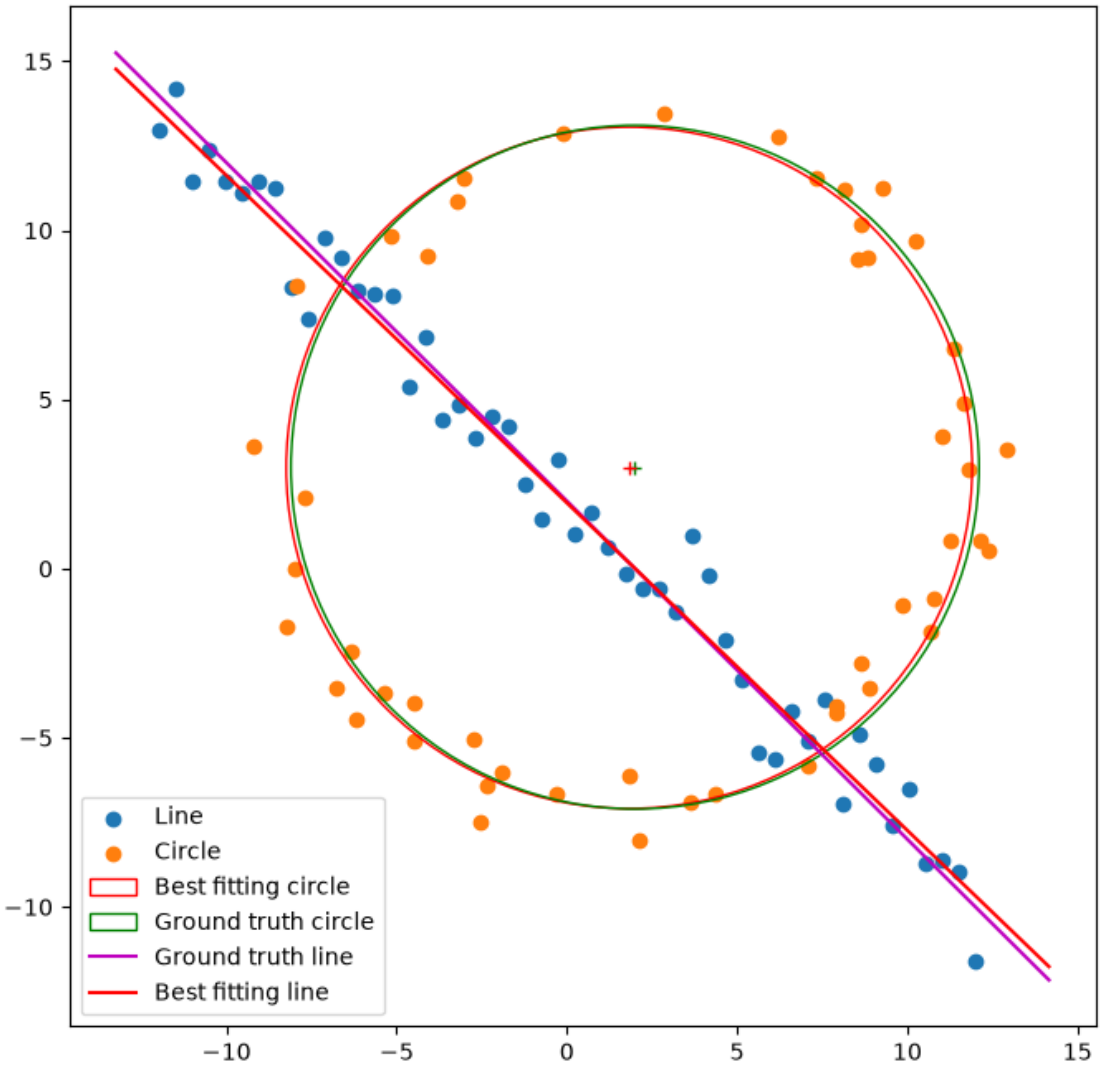
x_min, x_max = ax.get_xlim()
x_ = np.array([x_min, x_max])

y_ = m*x_ + b
y_pred = (d_best - a_best * x_) / b_best
circle_gt = plt.Circle((x0_gt, y0_gt), r, color='g', fill=False, label='Ground␣
 ↪truth circle')
circle_pred = plt.Circle((x0_best, y0_best), r_best, color='r', fill=False,␣
 ↪label='Best fitting circle')

ax.plot((x0_gt), (y0_gt), '+', color='g')
ax.plot((x0_best), (y0_best), '+', color='r')
ax.add_patch(circle_pred)
ax.add_patch(circle_gt)
```

```
plt.plot(x_, y_, color='m', label='Ground truth line')
plt.plot(x_, y_pred, color='red', label='Best fitting line')

plt.legend()
plt.show()
```



If the circle is fitted first, RANSAC may select three points from the line, producing a circle with a very large radius. Since a small arc of a large circle closely resembles a straight line, many line points may fall within the circle's inlier threshold. Consequently, the circle model may incorrectly classify line points as inliers.

Question 03

[ ]:
```
# Compute a homography from 4 point correspondences and warp the overlay image.
def compute_homography(src_pts, dst_pts):
    A, b = [], []
    for (x, y), (u, v) in zip(src_pts, dst_pts):
        A.append([x, y, 1, 0, 0, 0, -u * x, -u * y])
        b.append(u)
        A.append([0, 0, 0, x, y, 1, -v * x, -v * y])
        b.append(v)
    h = np.linalg.solve(np.asarray(A, dtype=np.float64), np.asarray(b, dtype=np.
 ↪float64))
    return np.append(h, 1).reshape(3, 3)


def apply_homography(background_bgr, overlay_bgr, picker_src, picker_dst):
    src_pts = picker_src.get()
    dst_pts = picker_dst.get()

    H = compute_homography(src_pts, dst_pts)
    h, w = background_bgr.shape[:2]
    warped_bgr = cv.warpPerspective(overlay_bgr, H, (w, h))

    mask = np.full(overlay_bgr.shape[:2], 255, dtype=np.uint8)
    warped_mask = cv.warpPerspective(mask, H, (w, h))
    warped_bgr = cv.warpPerspective(overlay_bgr, H, (w, h))
    result_bgr = background_bgr.copy()
    result_bgr[warped_mask > 0] = warped_bgr[warped_mask > 0]

    fig, ax = plt.subplots(1, 2, figsize=(8, 5))
    ax[0].imshow(background_rgb)
    ax[0].set_title("Background Image")
    ax[0].axis("off")

    ax[1].imshow(overlay_rgb)
    ax[1].set_title("Overlay Image")
    ax[1].axis("off")
    plt.show()

    fig, ax = plt.subplots(1, 1, figsize=(8, 5))
    result_rgb = cv.cvtColor(result_bgr, cv.COLOR_BGR2RGB)
    ax.imshow(result_rgb)
    ax.set_title("Homography Overlay")
    ax.axis("off")
    plt.show()


class PointPicker:
    def __init__(self, image_rgb, title, n_points=4):
```

```
        self.n = n_points
        self.title = title
        self.pts = []
        self.fig, self.ax = plt.subplots(figsize=(10, 6))
        self.ax.imshow(image_rgb)
        self.ax.set_title(f"{title} - click {n_points} points in order")
        self.ax.axis("off")
        self.marks = self.ax.plot([], [], "r+", markersize=14, mew=2)[0]
        self.cid = self.fig.canvas.mpl_connect("button_press_event", self.
↪on_click)
        plt.show()

    def on_click(self, event):
        # Ignore clicks outside the image, and while zoom/pan is active
        if event.inaxes != self.ax or len(self.pts) >= self.n:
            return
        tb = getattr(self.fig.canvas, "toolbar", None)
        if tb is not None and tb.mode != "":
            return
        self.pts.append((event.xdata, event.ydata))
        self.ax.text(event.xdata + 5, event.ydata - 5, str(len(self.pts)),
                     color="yellow", fontsize=14, weight="bold")
        xs, ys = zip(*self.pts)
        self.marks.set_data(xs, ys)
        if len(self.pts) == self.n:
            self.fig.canvas.mpl_disconnect(self.cid)
            self.ax.set_title(f"{self.title}")
        self.fig.canvas.draw_idle()

    def get(self):
        if len(self.pts) != self.n:
            raise ValueError(f"Only {len(self.pts)}/{self.n} points selected.")
        return np.asarray(self.pts, dtype=np.float32)
```

First pair of images

[ ]:
```
# Pick matching corners on the overlay and the background image.
%matplotlib widget

background_bgr = cv.imread("./a2images/billboard_road.jpg")
overlay_bgr = cv.imread("./a2images/veil_nebula.jpg")
background_rgb = cv.cvtColor(background_bgr, cv.COLOR_BGR2RGB)
overlay_rgb = cv.cvtColor(overlay_bgr, cv.COLOR_BGR2RGB)

picker_src = PointPicker(overlay_rgb, "Overlay image")
picker_dst = PointPicker(background_rgb, "Background image")
```

[12]:
```
%matplotlib inline
plt.close('all')
apply_homography(background_bgr, overlay_bgr, picker_src, picker_dst)
```



What's more beautiful than a fragment of a long-dead star lighting up a roadside billboard? The Veil Nebula, a remnant of a supernova that once tore a star apart, brings a glimpse of the distant cosmos into an ordinary scene on Earth.

Second pair of images

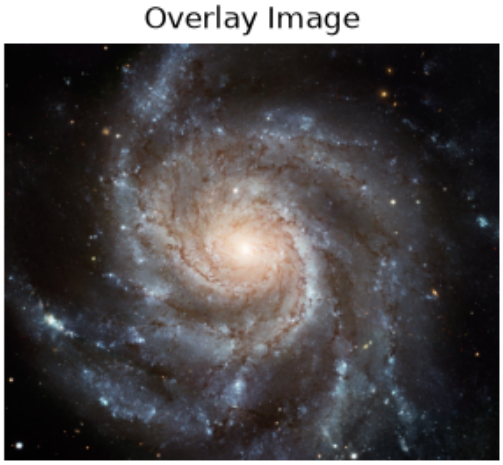
[ ]:
```
# Pick matching corners for the second image pair.
%matplotlib widget

background_bgr = cv.imread("./a2images/forest_cabin.jpg")
overlay_bgr = cv.imread("./a2images/messier101.jpg")
background_rgb = cv.cvtColor(background_bgr, cv.COLOR_BGR2RGB)
overlay_rgb = cv.cvtColor(overlay_bgr, cv.COLOR_BGR2RGB)

picker_src = PointPicker(overlay_rgb, "Overlay image")
picker_dst = PointPicker(background_rgb, "Background image")
```

[16]:
```
%matplotlib inline
plt.close('all')
apply_homography(background_bgr, overlay_bgr, picker_src, picker_dst)
```







Deep within a quiet forest, a glimpse of a distant galaxy appears across the wooden ceiling of a secluded cabin. Messier 101, with its delicate spiral arms and luminous blue and white hues, brings the vast beauty of the cosmos into the warmth of a natural retreat.

Question 04

[ ]:
```
# Match SIFT features and align each PCB image to the reference template.
template = cv.imread("./a2images/pcbs/AECC1.jpg")
template_gray = cv.cvtColor(template, cv.COLOR_BGR2GRAY)

image_names = ["AAAA1", "ADAD1", "AECC2", "AECC3", "AECC4"]

sift = cv.SIFT_create()
bf = cv.BFMatcher()

# Detect SIFT features in the template
kp_template, des_template = sift.detectAndCompute(template_gray, None)

h, w = template_gray.shape[:2]


def align_and_compare(image_name, template, template_gray, kp_template,␣
 ↪des_template, sift, bf):
    pcb = cv.imread(f"./a2images/pcbs/{image_name}.jpg")
    if pcb is None:
```

```
        print(f"Could not load {image_name}.jpg")
        return None

    pcb_gray = cv.cvtColor(pcb, cv.COLOR_BGR2GRAY)

    # Detect SIFT features in the PCB image
    kp_pcb, des_pcb = sift.detectAndCompute(pcb_gray, None)
    if des_pcb is None or des_template is None:
        print(f"Could not detect sufficient features in {image_name}.")
        return None

    # Match descriptors: template -> PCB
    matches = bf.knnMatch(des_template, des_pcb, k=2)
    good_matches = [
        m for pair in matches
        if len(pair) == 2
        for m, n in [pair]
        if m.distance < 0.8 * n.distance
    ]

    if len(good_matches) < 4:
        print(f"Not enough good matches for {image_name}.")
        return None

    src_pts = np.float32([kp_pcb[m.trainIdx].pt for m in good_matches]).
↪reshape(-1, 1, 2)
    dst_pts = np.float32([kp_template[m.queryIdx].pt for m in good_matches]).
↪reshape(-1, 1, 2)

    # Estimate homography using RANSAC
    H, mask = cv.findHomography(
        src_pts, dst_pts, cv.RANSAC, 5.0
    )

    if H is None or mask is None:
        print(f"Homography could not be computed for {image_name}.")
        return None

    # Align PCB with the template
    aligned_pcb = cv.warpPerspective(pcb, H, (w, h))

    # Compute absolute pixel-wise difference
    diff = cv.absdiff(template, aligned_pcb)
    gray_diff = cv.cvtColor(diff, cv.COLOR_BGR2GRAY)

    # Draw RANSAC inlier matches
    matches_img = cv.drawMatches(
```

```
        template, kp_template,
        pcb, kp_pcb,
        good_matches, None,
        matchColor=(0, 255, 0),
        singlePointColor=None,
        matchesMask=mask.ravel().tolist(),
        flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    return {
        "aligned": aligned_pcb,
        "diff": diff,
        "gray_diff": gray_diff,
        "matches": matches_img
    }


# Process all PCB images
for image_name in image_names:
    result = align_and_compare(
        image_name, template, template_gray,
        kp_template, des_template, sift, bf
    )

    if result is None:
        continue

    fig, ax = plt.subplots(1, 3, figsize=(16, 5))

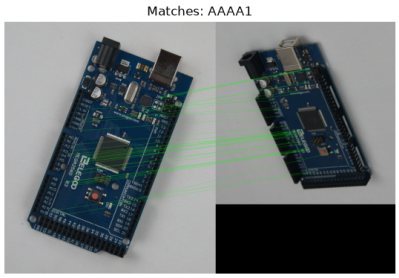
    ax[0].imshow(cv.cvtColor(result["matches"], cv.COLOR_BGR2RGB))
    ax[0].set_title(f"Matches: {image_name}")

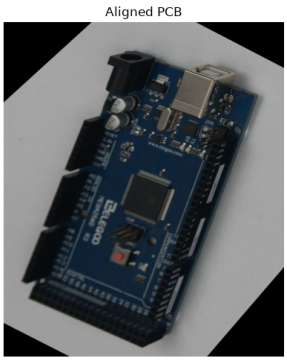
    ax[1].imshow(cv.cvtColor(result["aligned"], cv.COLOR_BGR2RGB))
    ax[1].set_title("Aligned PCB")

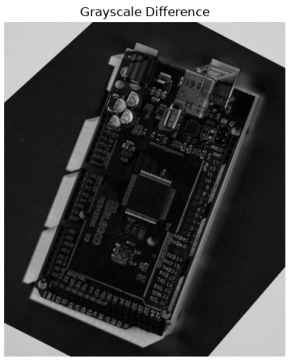
    ax[2].imshow(result["gray_diff"], cmap="gray")
    ax[2].set_title("Grayscale Difference")

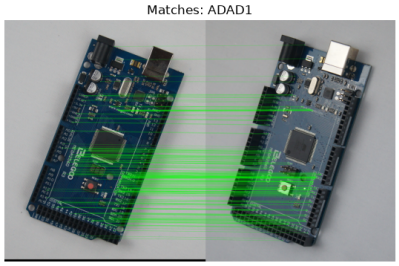
    for axis in ax:
        axis.axis("off")

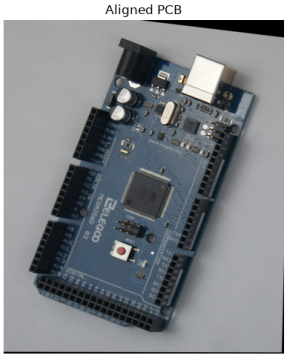
    plt.tight_layout()
    plt.show()
```

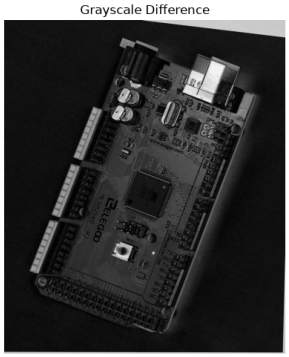

Matches: AAAA1

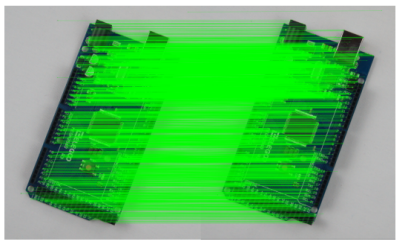

Aligned PCB

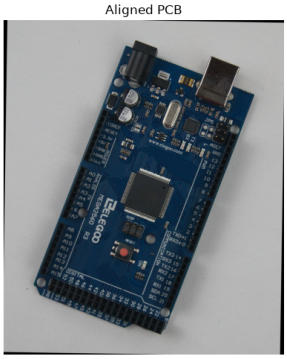

Grayscale Difference

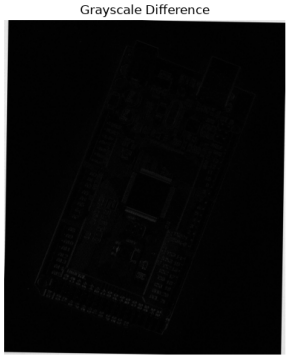

Matches: ADAD1

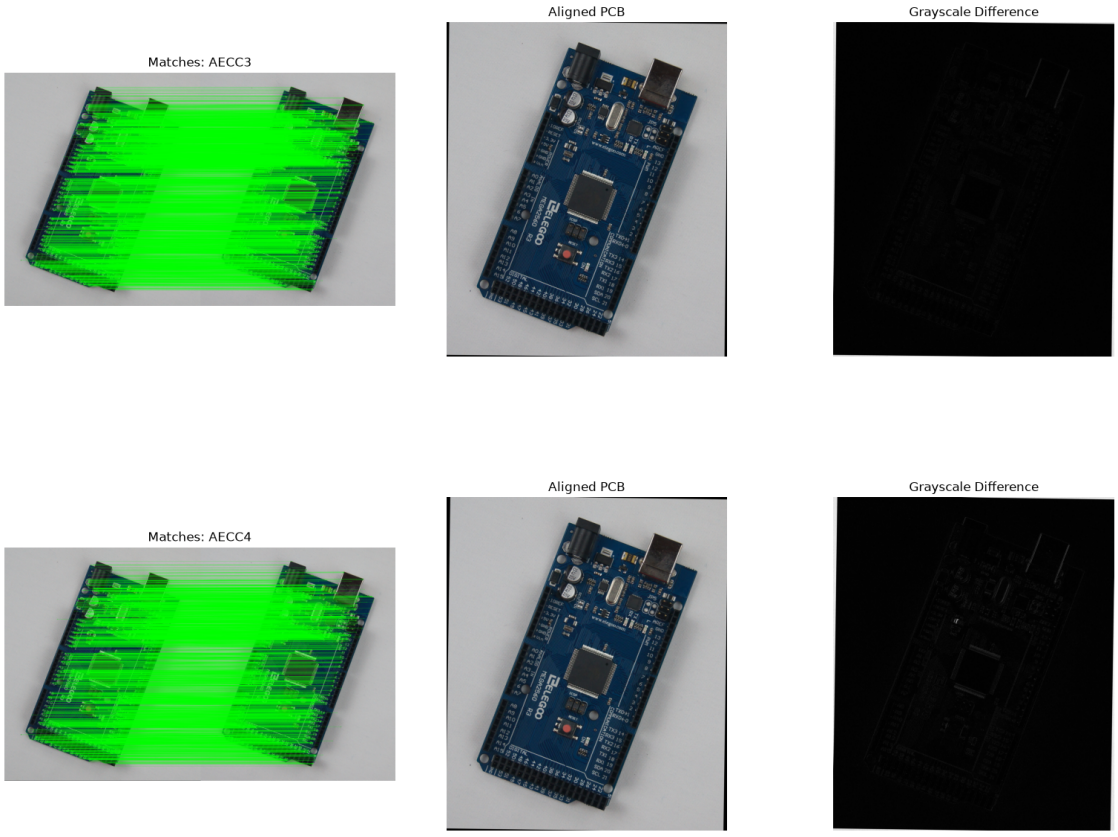

Aligned PCB


Grayscale Difference


Matches: AECC2


Aligned PCB


Grayscale Difference



The high pixel differences around the USB Type-B port, capacitors, and reset button are caused by changes in reflected light between the reference and PCB images. Since these components have metallic surfaces that reflect light directionally, differences in lighting or camera angle produce different pixel intensities even when the components are identical. As a result, the absolute pixel-wise difference highlights these regions.